

How to Run These Scripts

This is a simple guide for people who are not developers.

It answers three questions:

1. **What** each step does
2. **Where** it should be run
3. **How** to run it and what a successful result looks like

Note: This guide does not replace the detailed technical documents.

- For full technical steps, see [SCRIPT-TESTING-GUIDE.md](#)
- For product detail, see [README.md](#)
- For server names and folders, see [SERVERS.md](#)

What this tool is for (in one sentence)

These scripts check that the software files on a server still match the version that was approved in UAT (test), and that the program still looks healthy.

Golden rules (read before anything else)

1. **Only use Windows PowerShell 5.1** - not "PowerShell 7" or other terminals.
 2. **A pass is only the number 0.** Any other number means stop.
 3. **If a blank is still filled with REPLACE or TBD, stop.** Get the real value from your release owner or server owner first.
 4. **Do not run production steps without permission** and a scheduled change window.
 5. **Never paste passwords, keys, or secret values** into tickets, email, or chat.
 6. **If you are unsure, stop and ask.** That is always the correct action.
-

Who does what

Role	When they are needed
You (tester / operator)	Run the steps, record results, stop on failure
Release owner	Must be present for UAT baseline capture; helps on production checks
Server owner	Must approve/ support production runs
Repository maintainer/ tech lead	Helps if automated tests fail or commands are unclear

Where things run (map)

Think of three places. You will move between them in order.

Place	What kind of computer	What happens there
A. Your workstation	Your normal work PC	Practice / automated tests only -doesnotchangeNe□
B. UAT server	Test environment box (for outbound processors, often VESMSEGRESSUAT)	Create the official "approved" snapshot after UAT sign-off
C. Target / production server	The live (or target) box for that system	Compare files, check config, check health

Typical path for one release check:

Workstation (A) → UAT (B) → Target server (C)
 tests OK save baseline prove match+ health

You usually need a **remote desktop (RDP)** session to UAT and production servers. Use the access method your organization already approved.

What "success" and "failure" mean

After every command, look at the **exit number** (also called exit code).

In PowerShell, type this right after the script finishes:

```
$LASTEXITCODE
```

Numb er	Plain meaning	What you do
0	Pass - everything checked out	Write it down; continue to the next step
1	Fail - files or settings do not match	Stop. Escalate.
2	Error - could not trust or reach the baseline/ cloud check	Stop. Escalate.
3	Fail - program did not look healthy	Stop. Escalate.
10	Error - command was incomplete or unsafe	Fix the filled-in values with your lead, then try again

Only 0 means pass.

Before you start: fill this form

Do **not** invent folder names. Ask the release owner or use the official deployment runbook / [SERVERS.md](#).

#	What you need	Who usually has it	Write it here
1	Name of the system/ processor	Release owner	
2	Folder of the approved UAT files	Release owner	
3	Folder of the target/ production files	Server owner	
4	Path to the baseline file (ends in .json)	Release owner	
5	Cloud trust parameter name (starts with /ves/...)	Release owner/ cloud contact	
6	Release tag (looks like Name/vl. 4. 0)	Release owner	
7	Approved commit or label text	Release owner	
8	Cloud region (e.g. us-gov-west-1 or us-gov-east-1)	Cloud / ops contact	
9	Where the scripts are installed on the server (often D: \ ves-verify)	Server owner	
10	Change ticket/ authorization number (for production)	Change management	

If any box for the step you are about to run is empty --. **stop.**

How to open the right window (every machine)

Do this at the start on **every** computer you use for this work.

Step A - Open the correct PowerShell

1. On the Windows Start menu, search for **Windows PowerShell**.
2. Open **Windows PowerShell** (not "PowerShell 7", not "ISE" unless your lead says otherwise).
3. You should see a blue or black window that accepts typed commands.

Step B - Go to the script folder

Type the path your lead gave you, then press Enter.

On many servers:

```
Set-Location 'D:\ves-verify'
```

On a personal workstation, use the folder where this project was copied, for example:

```
Set-Location 'C:\Users\howardr01\Post-Deployment'
```

Step C - Confirm PowerShell version

```
$PSVersionTable.PSVersion
```

The first number (Major) should be **5**.

If it is **7**, close the window and open Windows PowerShell 5.1 instead.

Step D -Write down the version stamp (optional but recommended)

```
git rev-parse --short HEAD
```

Copy that short code into your test notes.

Main path (do these in order)

These are the steps most testers use to validate a release.

Step 1 - Automated check on your PC (optional but recommended)

Where	Your workstation only - not production
What	Confirms the checking tools themselves still work
Script	Invoke-Tests.ps1
Risk	Safe - does not change servers

How

1. Open PowerShell and go to the project folder (see above).
2. Copy and paste this, then press Enter:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\Invoke-Tests.ps1  
$LASTEXITCODE
```

What good looks like

- The window shows tests running.
- The last number printed is **0** .

If it is not 0

Stop. Contact the repository maintainer. Do not go to UAT or production.

Step 2 - Readiness check *C'are we set up correctly?*¹¹

Where	UAT or the machine that can reach cloud parameters and the baseline file
What	Confirms cloud access and baseline files are ready- does not change application files
Script	Invoke-Preflight.ps1
Risk	Safe (read-only)

How

1. Get the filled-in command from your release owner, **or** use the pattern below and replace every REPLACE ... with real values.
2. Run:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass,  
-File .\Invoke-Preflight.ps1'  
-Processor REPLACE_WITH_PROCESSOR_NAME'  
-ApprovedCommitParam REPLACE_WITH_APPROVED_COMMIT_PARAM'  
-TrustParam REPLACE_WITH_SSM_TRUST_PARAM'  
-ManifestPath REPLACE WITH BASELINE MANIFEST PATH'  
-Region REPLACE_WITH_CONFIRMED_REGION,  
-Json  
$LASTEXITCODE
```

What good looks like

- Exit code **0** ("ready").

If it is not 0

Stop. Usually this means login, region, or file path is wrong. Escalate to the release owner or cloud contact. Do not capture a baseline yet.

Step 3 - Save the official UAT baseline {after UAT is approved)

Where	UAT server/ UAT approval host
What	Takes a fingerprint of the approved files and stores a trusted record
Script	Invoke-Verification.ps1 with mode Capture
Risk	Writes the baseline record(and related trust data)- release owner must be present
When	Only after UAT sign-off is complete

How

1. Confirm the **release owner is with you** (in person or on a call).
2. Confirm UAT was signed off.
3. Run the capture command your owner provides (pattern below):

```
powershell.exe -NoProfile -ExecutionPolicy Bypass,
-File .\Invoke-Verification.ps1'
-Mode Capture'
-ReleaseRoot REPLACE WITH UAT APPROVED RELEASE FOLDER,
-ManifestPath REPLACE_WITH_BASELINE_MANIFEST_PATH'
-TrustParam REPLACE_WITH_SSM_TRUST_PARAM'
-ArchiveRepo REPLACE_WITH_BASELINE_GIT_CHECKOUT,
-ReleaseTag REPLACE_WITH_RELEASE_TAG'
-Processor REPLACE_WITH_PROCESSOR_NAME,
-CommitSha REPLACE_WITH_APPROVED_COMMIT'
-Environment uat'
-Region REPLACE_WITH_CONFIRMED_REGION'
```


-Json

\$LASTEXITCODE

What good looks like

- Exit code **0** .
- You can write down: release tag, baseline file path, and any log path shown on screen.

If it is not 0

Stop. Do not proceed to production comparison.

What you must write down from this step

- Date and time
- Server name
- Release tag
- Baseline file path
- Exit code
- Path to the log file if shown

Step 4 - Compare the target server files to the UAT baseline

Where	The target/ production server for that system (or the approved runner that can see those folders)
What	Checks that live files still match the approved UAT snapshot
Script	Invoke-Verification.psl with mode VerifyFiles
Risk	Safe check (read-only), but production access needs approval
When	After a deploy, or when validating current production

How

- 1. Confirm you have **change/ access authorization** for that server.
- 2. Confirm release owner and server owner are aware.
- 3. Run:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass,
-File .\Invoke-Verification.psl'
-Mode VerifyFiles'
-ReleaseRoot REPLACE_WITH_TARGET_RELEASE_FOLDER'
-ManifestPath REPLACE_WITH_BASELINE_MANIFEST_PATH'
-TrustParam REPLACE_WITH_SSM_TRUST_PARAM,
-Processor REPLACE_WITH_PROCESSOR_NAME'
-Environment prod'
-Region REPLACE_WITH_CONFIRMED_REGION'
-Json
$LASTEXITCODE
```

What good looks like

- Exit code 0 - files match.

If it is not 0

Number	Plain English
1	Something is missing, changed, or extra compared to UAT
2	Could not confirm the trusted baseline
10	The command was incomplete or wrong

Stop and escalate. Attach the command (with secrets removed) , exit code, and any lines that say MISSING , CHANGED , or EXTRA .

Step 5 - Check configuration settings (if your team uses a contract)

Where	Same target server as Step 4
What	Checks important settings in config files(paths that are allowed to differ by environment are handled by the contract)
Script	Invoke-Verification.psl with mode VerifyConfig (or All for files + config together)
Risk	Safe check(read-only)
When	When your release package includes a config contract file

How (config only)

```
powershell.exe -NoProfile -ExecutionPolicy Bypass'
-File .\Invoke-Verification.psl'
-Mode VerifyConfig'
-ConfigContract REPLACE_WITH_CONFIG_CONTRACT_PATH,
-ConfigPath REPLACE_WITH_LIVE_CONFIG_PATH'
-TrustParam REPLACE_WITH_SSM_TRUST_PARAM'
-Processor REPLACE_WITH_PROCESSOR_NAME'
-Environment prod'
-Region REPLACE_WITH_CONFIRMED_REGION'
-Json
$LASTEXITCODE
```

What good looks like

- Exit code 0 .

If it is not 0

Stop and escalate the same way as Step 4.

If your lead says "we are not using config contracts for this system," skip this step and note that in your sign-off form.

Step 6 - Health check (is the program actually running?)

Where	The target server
What	Confirms the program is running the way operations expects (process, scheduled task, recent log activity, etc.)
Script	Invoke-HealthCheck.ps1
Risk	Safe check (read-only)
When	After files (and config, if used) already passed

How

Use the exact command your release owner provides for that processor.
A typical outbound-processor pattern looks like this:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass'  
-File .\Invoke-HealthCheck.ps1'  
-Processor REPLACE_WITH_PROCESSOR_NAME'  
-ProcessPathRoot REPLACE_WITH_TARGET_RELEASE_FOLDER'  
-ProcessArgumentPattern 'REPLACE_WITH_MODE_REGEX' '  
-ScheduledTasks REPLACE_WITH_TASK_NAME'
```

```
-FreshLogDir REPLACE_WITH_LOG_DIR'  
-FreshLogMaxAgeMinutes 60,  
-RequiredAssemblies REPLACE WITH EXE PATH'  
-Environment prod'  
-Json  
$LASTEXITCODE
```

What good looks like

- Exit code **0** .

If it is not 0

- **3** means health failed (not running the way it should).
- **10** often means the command was missing required checks.

Stop and escalate.

Optional paths (only if someone assigns them to you)

These are **not** part of the normal "prove the release matches" tester path. Do them only when ops or change management asks.

Optional A - Trial deploy check (no files copied)

Where	Usually UAT first
What	Asks "would this deploy be allowed?" without copying files

How	Run the deploy script with -Whatif
Risk	Low if you use -WhatIf ; still needs permission

Example shape (UAT DBQ pilot wrapper):

```
.\processors\Deploy-0utboundDBQ-uat.psl'
  -StagedRoot D:\stage\0utboundDBQ'
  -StagedCommit REPLACE_WITH_STAGED_COMMIT'
  -ConfirmedRunbookValues'
  -WhatIf
```

Expected: exit code **0** for a clean trial.

If not 0, do **not** run a real deploy.

Optional B - Real deploy (copies files)

Where	Target server named in the change ticket
What	Stops the old instance, backs up, copies new files, restarts, then verifies
Risk	High - changes the live system
Required	Formal change authorization, release owner, server owner

Only run the real deploy command your lead gives you (without **-Whatif**).

If you are not sure whether **-WhatIf** is present, **do not run it**.

Optional C - Drift check (ongoing monitoring)

Where	Ops runner machine
What	Re-checks many systems on a schedule
Who	Usually operations , not a one-time tester
Note	Inventory must be complete before this can claim full coverage

If asked to run a one-time drift check, ops will give you the exact command. Typical shape:

```
.\Start-DriftRunner.psl -TargetsFile D:\ves-verify\targets.json
```

Which script goes on which machine (quick cheat sheet)

Script (short name)	Workstation	UAT	Production / target	Safe for beginners?
Automated tests (Invoke-Tests. psl)	Yes	No	No	Yes
Preflight (Invoke-Preflight. psl)	Sometimes	Yes	Sometimes	Yes (read-only)
Capture baseline (... -Mode Capture)	No*	Yes	No	Only with release owner
Compare files (... -Mode VerifyFiles)	No*	Yes	Yes	Yes with approval

Script (short name)	Workstation	UAT	Production / target	Safe for beginners?
Check config (... -Mode VerifyConfig)	No*	Yes	Yes	Yes with approval
Health (Invoke-HealthCheck. ps1)	No*	Yes	Yes	Yes with approval
Deploy trial (Deploy-... - Whatif)	No	Prefer	Only if assigned	Only if assigned
Real deploy (Deploy-...)	No	Pilot first	Only with change ticket	No - lead only
Drift runner	No	Ops	Ops	Ops only

* Some checks can be run from a central runner if it can see the folders; your team will tell you if that applies.

What to write down every time (your evidence)

Copy this into a ticket, spreadsheet, or paper log for **each** run:

Date/ time (include time zone if you know it):

Computer/ server name:

Step number (1-6 or optional):

Script name:

Release/ processor name:

Release tag:

Authorization/ ticket number:

Exit code:

Pass or Fail:

Log file path (if shown):

Any error text (do not include secrets):

Your name:

If something fails - what to hand over

Send this package to the release owner or on-call contact:

1. Which step you were on
2. Server name and date/time
3. Exit code
4. The command you ran (**remove any secrets first**)
5. Screenshot of the end of the PowerShell window
6. Log file path if the screen printed one
7. Any lines that say MISSING , CHANGED , EXTRA , or health failure reasons

Then **wait**. Do not try random different commands.

Final sign-off form

Field	Fill in
Tester name	
Date	
System / processor	
Release tag	

Field	Fill in
Step 1 (workstation tests) result	Pass/ Fail/ Skipped
Step 2 (preflight) result	Pass/ Fail/ Skipped
Step 3 (UAT capture) result	Pass/ Fail/ Skipped
Step 4 (file compare) result	Pass/ Fail
Step 5 (config) result	Pass/ Fail/ Not used
Step 6 (health) result	Pass/ Fail
Final overall result	PASS only if all required steps were 0
Release owner initials	
Server owner initials (prod)	
Notes	

Super-short version (wall chart)

1. Open **Windows PowerShell 5.1**.
2. Go to the script folder.
3. On your **PC**, run tests if asked --. need **0**.
4. On **UAT**, run preflight --. need **0**.
5. On **UAT**, with the release owner, capture the baseline --. need **0**.
6. On the **target server**, compare files --. need **0**.
7. If used, check config --. need **0**.
8. Check health --. need **0**.
9. Write down results.
10. Any number other than **0** = **stop and escalate**.

Helpful tips

- **Copy and paste commands carefully.** One wrong folder name can cause a failure that looks scary but is just a typo.
 - **Keep one PowerShell window open** for the whole session.
 - **Do not close the window** until you have copied the exit code and any log path.
 - **Ask for a filled-in command** if placeholders confuse you - that is normal.
 - **Deploy and drift work are optional** and usually owned by operations or a technical lead.
-

Where to get more detail (if you want it later)

Document	Audience
How-to-Run-Scripts.md (this file)	Non-technical/ first-time operators
SCRIPT-TESTING-GUIDE.md	Testers who want full technical command detail
README.md	Full product explanation
SERVERS.md	Server names and folder map

Tester system runbook

Step-by-step guide for testers who run the post-deployment scripts **on OMS systems** (workstation -+ UAT -+ production target servers).

Purpose: prove that what runs on a target system matches the UAT-approved release, and stop cleanly when files, config, trust, or health fail.

Runtime: Windows PowerShell **5.1** only (powershell.exe , not PowerShell 7).

Authoritative detail: [README.md](#) (exit codes, trust model), [SERVERS.md](#) (server and processor path map). This guide is the ordered run path; those docs are the deep reference.

If you want a plain-language walkthrough, start with [RUNBOOK-NON-TECHNICAL.md](#).

Scripts a tester may run

Script	What it does	Where to run	Risk
Invoke-Tests.ps1	Automated file-match unit tests (Pester 5)	Workstation/CI	None (local fixtures only)
Invoke-Preflight.ps1	Read-only readiness: PS version, AWS CLI, SSM, baseline	UAT, runner, or target	Read-only
Invoke-Verification.ps1 - Mode Capture	Build approved UAT baseline + pin hash to SSM	UAT approval host	Writes manifest, SSM, optional Git archive
Invoke-Verification.ps1 - Mode VerifyFiles	Compare live files to trusted baseline	Target server	Read-only check

Script	What it does	Where to run	Risk
Invoke-Verification.ps1 - Mode VerifyConfig	Config contract check (keys/values vs contract+ SSM)	Target server	Read-only check
Invoke-Verification.ps1 - Mode All	VerifyFiles + VerifyConfig in one run	Target server	Read-only check
Invoke-HealthCheck.ps1	Assembly/ process/ task/ log/ endpoint probes	Target server	Read-only check
processors\Deploy-*.ps1 -Whatif	Pre-deploy gate only (no copy)	Target server (pilot)	Read-only gate
processors\Deploy-*.ps1	Gate -+ stop -+ backup -+ copy -+ restart -+ verify -+ health	Target server (pilot)	Write path- change control required
Start-DriftRunner.ps1	Re-verify every target in inventory	Ops runner	Read-only checks; needs complete inventory
tail-DriftTask.ps1	Register scheduled runner + heartbeat watchdog	Ops (elevated)	Writes scheduled tasks

Default tester path for release validation: Steps 0-5 below (tests -+ preflight-+ capture -+ file verify-+ config -+ health). Deploy and drift are optional and higher risk.

Approvals (do not skip)

Activity	Required approver
Local automated tests (Invoke-Tests.ps1)	Repository maintainer (or self for pure local work)
Preflight	Tester + server owner if run on shared hosts
UAT baseline capture (-Mode Capture)	Release owner present during the run
Production VerifyFiles / VerifyConfig / Health	Release owner and server owner
Deploy (Deploy-*.psl without -Whatif)	Formal change authorization
Any production execution window	Formal change authorization

If a value is blank or still shows REPLACE / TBD, **stop**.

Prerequisites

1. Open **Windows PowerShell 5.1** (not PowerShell 7).
2. Set location to the repository checkout on that machine:

Set-Location 'C:\Users\howardr01\Post-Deployment'

or the approved path on the server, e.g. D:\ves-verify

3. Confirm version:

```
$PSVersionTable.PSVersion
```

```
# Major should be 5
```

4. Record the commit under test:

```
git rev-parse --short HEAD
```

5. **Workstation only (Step 0):** Pester 5.x once:

```
Install-Module Pester -MinimumVersion 5.5.0 -Scope CurrentUser -Force -SkipPu
```



6. **SSM-dependent steps (1-4):** AWS CU installed; credentials or instance profile can
 `ssm:GetParameter` (and `ssm:PutParameter` on capture hosts) plus `kms:Decrypt` for
 SecureString parameters in the confirmed GovCloud region.

7. Optional durable logs (recommended before production):

```
$env:VES_AUDIT_LOG_DIR = '\\audit-share\ves-verify\logs'  
# or a local path such as D:\ves-verify\logs
```

If unset, scripts fall back to `%ProgramData%\ves-verify\logs` .

Keep the **same PowerShell window** for a session so `$LASTEXITCODE` and env vars stay
consistent.

Values worksheet {fill before Step 1)

Pull these from the Outbound Deployment Steps runbook and [SERVERS.md](#). Do not invent
production paths.

#	Value	Example shape	Your value
1	Processor name	OutboundDBQ	
2	UAT approved release folder (- ReleaseRoot for Capture)	C:\VLER_TEST_OUTBOUND\Processor s\VES.OutboundProcessor	

#	Value	Example shape	Your value
3	Production (or target) folder (- ReleaseRoot for Verify)	From runbook per server	
4	Baseline manifest path	D:\baselines\OutboundDBQ.json	
5	SSM trust parameter	/ves/OutboundDBQ/baseline-hash	
6	SSM approved-commit parameter	/ves/OutboundDBQ/approved-commit	
7	Release tag	OutboundDBQ/v1.4.0 (<system>/vMAJOR.MINOR.PATCH)	
8	Approved commit/ label string	Git SHA or TFS label	
9	GovCloud region	Confirm us-gov-west-1 vs us-gov-east-1	
10	Archive Git checkout (- ArchiveRepo for Capture)	D:\ves-verify	
11	Config contract path (if used)	D:\baselines\OutboundDBQ.config.json	
12	Live config path (if used) ☐	... \VES.OutboundDBQProcessor.exe.config	
13	Scheduled task name(s) for health	From runbook	
14	Fresh log directory for health	From runbook	

#	Value	Example shape	Your value
1	Process mode arg pattern	e.g. \bRTPDP\b for DBQ	
5			

Where processors live (quick map)

Outbound processors share one EXE name (VES.OutboundDBQProcessor.exe) and are distinguished by **folder + mode argument**, not process name alone. See [SERVERS.md](#) for full paths.

Tier	Server	Processors
UAT	VESMSEGRESSUAT	Ack, DBQ, XML (all on one box)
PROD	VESEMSEGRESS01	XML/ Outbound Events
PROD	VESEMSEGRESS02	Ack, DBQ XML
PROD	VESEMSEGRESS03	XML, DBQ

Processor	Mode arg	Typical <u>health identity</u>
Ack	RTP	Working dir under Ack tree + arg pattern \bRTP\b
DBQ	RTPDP	Working dir under DBQ tree + \bRTPDP\b
XML	RTP	Working dir under XML tree + folder/batch identity

Inbound and Citrix targets follow the same script pattern once paths are confirmed. Do not set targets.json inventoryComplete to true until every required server (including Citrix) is

Step-by-step run order

Step 0 - Automated tests (workstation only)

Confirms the file-match logic still works before any server work.

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\Invoke-Tests.psl  
$LASTEXITCODE
```

Expected: 0 (failed-test count).

If not 0 , **stop**. Do not continue to systems.

Step 1 - Preflight (UAT or runner; read-only)

Confirms PowerShell host, AWS CLI, SSM parameters, and baseline integrity. Touches **no** production or staged files.

Single processor:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass'  
-File .\Invoke-Preflight.psl'  
-Processor REPLACE_WITH_PROCESSOR_NAME'  
-ApprovedCommitParam REPLACE_WITH_APPROVED_COMMIT_PARAM'  
-TrustParam REPLACE_WITH_SSM_TRUST_PARAM'  
-ManifestPath REPLACE_WITH_BASELINE_MANIFEST_PATH'  
-Region REPLACE_WITH_CONFIRMED_REGION'  
-Json  
$LASTEXITCODE
```

Or every inventory target (when inventory is filled in):

```
powershell.exe -NoProfile -ExecutionPolicy Bypass'  
-File .\Invoke-Preflight.psl'  
-TargetsFile .\targets.json'
```

```
-Region REPLACE_WITH_CONFIRMED_REGION'
-Json
$LASTEXITCODE
```

Expected: 0 = ready. 2 = not ready (SSM, baseline, inventory, or host).

A manifest-pattern **WARN** means that baseline may need re-capture after exclude-rule changes; WARN alone does not fail preflight. See README "Upgrading: re-pin baselines..."

If not 0, **stop** and fix trust/region/paths before Capture or Verify.

Step 2 - Capture approved UAT baseline (UAT; release owner present)

Run only when UAT sign-off is complete. This creates the trusted baseline.

```
powershell.exe -NoProfile -ExecutionPolicy Bypass,
-File .\Invoke-Verification.ps1'
-Mode Capture,
-ReleaseRoot REPLACE_WITH_UAT_APPROVED_RELEASE_FOLDER'
-ManifestPath REPLACE_WITH_BASELINE_MANIFEST_PATH'
-TrustParam REPLACE_WITH_SSM_TRUST_PARAM,
-ArchiveRepo REPLACE_WITH_BASELINE_GIT_CHECKOUT'
-ReleaseTag REPLACE_WITH_RELEASE_TAG'
-Processor REPLACE_WITH_PROCESSOR_NAME'
-CommitSha REPLACE_WITH_APPROVED_COMMIT'
-Environment uat,
-Region REPLACE_WITH_CONFIRMED_REGION'
-Json
$LASTEXITCODE
```

Optionally add -PushRemote so the archive commit and tag leave the capture host immediately (failed push fails the capture).

Also pin the approved commit/label if your process requires it:

```
aws ssm put-parameter --name REPLACE_WITH_APPROVED_COMMIT_PARAM -
--value REPLACE_WITH_APPROVED_COMMIT -
--type SecureString --overwrite --region REPLACE_WITH_CONFIRMED_REGION
```

Expected: 0 .

If not 0 , **stop**.

Do not use -AllowUntrustedCapture or -AllowUnarchivedCapture for a real approved release(local tests only).

Save from this step:

- Release tag
- Manifest path
- Manifest hash shown in output
- JSONL log path
- Commit/ label pinned to SSM

Step 3 -Verify production (or target) files match baseline

Main content check. Run **on the target server** (or against that server's release root if your ops model allows a central runner with path access).

From local manifest + SSM trust:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -
-File .\Invoke-Verification.psl -
-Mode VerifyFiles -
-ReleaseRoot REPLACE_WITH_TARGET_RELEASE_FOLDER -
-ManifestPath REPLACE_WITH_BASELINE_MANIFEST_PATH -
-TrustParam REPLACE_WITH_SSM_TRUST_PARAM -
-Processor REPLACE_WITH_PROCESSOR_NAME -
-Environment prod -
-Region REPLACE_WITH_CONFIRMED_REGION -
```

```
-Json
$LASTEXITCODE
```

From Git release tag {SSM pin still applies when both are set):

```
powershell.exe -NoProfile -ExecutionPolicy Bypass'
-File .\Invoke-Verification.psl'
-Mode VerifyFiles'
-ReleaseRoot REPLACE_WITH_TARGET_RELEASE_FOLDER'
-BaselineRepo REPLACE_WITH_BASELINE_GIT_CHECKOUT'
-ReleaseTag REPLACE_WITH_RELEASE_TAG,
-TrustParam REPLACE_WITH_SSM_TRUST_PARAM'
-Processor REPLACE_WITH_PROCESSOR_NAME'
-Environment prod'
-Region REPLACE_WITH_CONFIRMED_REGION'
-Json
$LASTEXITCODE
```

Expected: 0.

Exit	Meaning	Action
0	Files match trusted baseline	Continue to config/health as planned
1	Drift(MISSING , CHANGED , EXTRA)	Stop and escalate
2	Trust/baselinefailure	Stop and escalate
10	Bad/incomplete command	Fix inputs and rerun

Step 4 - Verify config contract (when a contract exists}

Config files(*. config) are **excluded** from file-hash compare on purpose(environment-specific paths). Check them with the contract.

```
powershell.exe -NoProfile -ExecutionPolicy Bypass'
-File .\Invoke-Verification.psl'
-Mode VerifyConfig'
-ConfigContract REPLACE_WITH_CONFIG_CONTRACT_PATH'
-ConfigPath REPLACE_WITH_LIVE_CONFIG_PATH'
-TrustParam REPLACE_WITH_SSM_TRUST_PARAM -
-Processor REPLACE_WITH_PROCESSOR_NAME'
-Environment prod'
-Region REPLACE_WITH_CONFIRMED_REGION'
-Json
$LASTEXITCODE
```

Or combine files + config:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -
-File .\Invoke-Verification.psl'
-Mode All'
-ReleaseRoot REPLACE_WITH_TARGET_RELEASE_FOLDER -
-ManifestPath REPLACE_WITH_BASELINE_MANIFEST_PATH'
-ConfigContract REPLACE_WITH_CONFIG_CONTRACT_PATH'
-ConfigPath REPLACE_WITH_LIVE_CONFIG_PATH'
-TrustParam REPLACE_WITH_SSM_TRUST_PARAM'
-Processor REPLACE_WITH_PROCESSOR_NAME -
-Environment prod'

-Region REPLACE_WITH_CONFIRMED_REGION'
-Json
$LASTEXITCODE
```

Expected: 0 . Same stop rules as Step 3(1 drift, 2 trust, 10 usage).

Contract shape: samP-le.config.json. Sensitive values are masked in reports; never paste secrets into the worksheet or ticket.

Step S - Health check (target server)

Proves the instance is alive after files/config match. An invocation with **no probes** exits 10 (not a false green).

Outbound console EXE (no HTTP endpoint) - match by folder + mode arg:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass-
-File .\Invoke-HealthCheck.ps1-
-Processor REPLACE_WITH_PROCESSOR_NAME-
-ProcessPathRoot REPLACE_WITH_TARGET_RELEASE_FOLDER-
-ProcessArgumentPattern 'REPLACE_WITH_MODE_REGEX'-
-ScheduledTasks REPLACE_WITH_TASK_NAME-
-FreshLogDir REPLACE_WITH_LOG_DIR-
-FreshLogMaxAgeMinutes 60-
-RequiredAssemblies REPLACE_WITH_EXE_PATH-
-Environment prod-
-Json
$LASTEXITCODE
```

Example shapes (confirm against runbook before use):

- DBQ mode pattern: \bRTPDP\b
- Ack/ XML mode pattern: often \bRTP\b (folder distinguishes them)

Expected: 0 . Health failure is 3 .

Java/gateway service profile (out of current OMS manual-copy scope; retained for later work) uses -ServiceName and -HealthUrl instead of process/task/log.

Step 6 - Optional: deploy pilot (UAT first; change control)

Use only with formal authorization. Prefer the thin wrapper for the box:

```
# Gate only - no copy
.\processors\Deploy-OutboundDBQ-uat.psl'
  -StagedRoot D:\stage\OutboundDBQ'
  -StagedCommit REPLACE_WITH_STAGED_COMMIT'
  -ConfirmedRunbookValues'
  -Whatif
```

```
# Real deploy after Whatif is clean
.\processors\Deploy-OutboundDBQ-uat.psl'
  -StagedRoot D:\stage\OutboundDBQ'
  -StagedCommit REPLACE_WITH_STAGED_COMMIT'
  -ConfirmedRunbookValues
```

-ConfirmedRunbookValues is required on the UAT DBQ wrapper until task name and log directory are confirmed against the runbook.

Deploy flow: gate -> stop matching instance -> backup -> copy -> restart task -> verify -> health.
Pilot on **UAT egress** before any PROD use.

Step 7 - Optional: drift runner (ops)

Inventory is **fail-closed**. The checked-in targets.json has inventoryComplete: false until Citrix and production paths are confirmed. The runner will not claim full coverage while incomplete.

```
# One-shot re-verify of inventory targets
.\Start-DriftRunner.psl -TargetsFile D:\ves-verify\targets.json

# Register scheduled runner+ independent heartbeat watchdog (elevated, ops)
.\Install-DriftTask.psl -TargetsFile D:\ves-verify\targets.json'
  -IntervalMinutes 30 -LogDir \\audit-share\ves-verify\logs
```

Telemetry push is **disabled** today; exit codes and JSONL logs are the only signal. Configure a durable log directory before production reliance.

Exit codes (plain language)

Cod e	Outcom e	Meaning	Action
0	PASS	Check succeeded	Record and continue or finish
1	FAIL	File or config drift	Stop and escalate
2	ERROR	Trust, inventory, baseline, or runtime failure	Stop and escalate
3	FAIL	Health probe failed	Stop and escalate
10	ERROR	Usage error or unsafe configuration	Fix inputs and rerun

A missing baseline, incomplete inventory, dead check, or empty health probe is **never** a pass.

What to record each time

For every run:

- 1. Script name, mode, date/time (UTC preferred)
- 2. Server or workstation name
- 3. `git rev-parse --short HEAD`
- 4. Processor, release tag, environment, region
- 5. Exact command used (**sanitize** secrets)
- 6. `$LASTEXITCODE`
- 7. Any `MISSING` , `CHANGED` , or `EXTRA` lines (or health failure reasons)

8. JSONL log path (console or default under %ProgramData%\ves-verify\logs / VES_AUDIT_LOG_DIR)

Never copy passwords, tokens, connection strings, or raw SSM secret values into the record.

Escalation packet

When a run does not end in 0, hand over:

- 1. Script name and sanitized command
- 2. Date/time and host name
- 3. Commit hash and release tag
- 4. Exit code
- 5. Full drift or failure lines
- 6. JSONL log file path
- 7. Screenshot of the final output section

Sign-off block

Field	Value
Tester	
Repository maintainer	
Release owner	
Server owner	
Date	
Steps completed (0-5 / 6 / 7)	
Final result (PASS / FAIL / ERROR)	

Field	Value
Exit code of last critical step	

Quick summary for non-technical operators

1. **Workstation:** run automated tests - must be 0 .
2. **UAT:** preflight, then capture the approved baseline with the release owner.
3. **Target system:** verify files (and config if used), then health.
4. A pass is only exit code 0 .
5. Any other number means **stop** and hand over the escalation packet.
6. Deploy and scheduled drift are optional, higher-risk, ops-owned paths.

Do nots

- Do not run under PowerShell 7 for production checks.
- Do not use untrusted/unarchived capture switches for real releases.
- Do not invent PROD paths; pull them from the runbook / SERVERS.md.
- Do not treat incomplete targets.json as full inventory coverage.
- Do not log secrets or paste SSM SecureString values into tickets.
- Do not skip formal change control for deploy (Step 6) or production windows.

Non-Technical Runbook: How to Run the Post-Deployment Scripts

This guide is for testers and operators who are not deeply technical. It explains **what to run**, **where to run it**, and **how to run it** in clear, simple language.

For detailed technical guidance, use [RUNBOOK.md](#).

1) What this process does

You are confirming that:

1. The script suite is healthy.
2. The approved UAT release was captured correctly.
3. Production files match what UAT approved.
4. Health checks pass after deployment.

If anything fails, stop and escalate.

2) Before you start (yes/no checklist)

Check each item before running anything:

- I am using **Windows PowerShell 5.1** (not PowerShell 7).
 - ☐ I am on the **correct machine** for this step (workstation, UAT host, or PROD server).
 - I know where the repo is on this machine (example: D: \ves-verify).
 - ☐ I have the required values (processor name, manifest path, SSM paths, release tag, region).
 - Required approvers are present for UAT capture or production actions.
-

3) Where each script is run

s c r i p t	where to run it	J Why
Invoke-Tests.ps1	Tester workstation	Make sure script logic is healthy first
Invoke-Preflight.ps1	The machine where you will run verification/deploy	Check PowerShell, AWS access, and baseline readiness
Invoke-Verification.ps1 -Mode Capture	UAT approval host	Save and lock approved UAT baseline
Invoke-Verification.ps1 -Mode VerifyFiles	Production target server	Check production files match approved baseline
Invoke-Verification.ps1 -Mode VerifyConfig	Production target server	Check config values are valid
Invoke-HealthCheck.ps1	Production target server	Confirm app/process is healthy
Invoke-PreDeployGate.ps1	Staging/deploy machine	Block unsafe deployment
processors\Deploy- <system>.ps1	Production target server	Perform controlled deploy
Start-DriftRunner.ps1	Ops runner or target server	Repeat drift checks on schedule
Install-DriftTask.ps1	Test-DriftHeartbeat.ps1	Ops runner or target server (admin)

Same machine as drift runner

Register scheduled checks

Confirm scheduled drift
checks are still running

4) Step-by-step (normal tester flow)

Follow these steps in order.

Step A - Open PowerShell and go to repo

Where: current machine for this step.

```
Set-Location '<path-to-repo-on-this-machine>'
# Example: D:\ves-verify
```

Then confirm PowerShell version:

```
$PSVersionTable.PSVersion
```

Success means Major version is 5.

Step B - Run unit tests first

Where: tester workstation.

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\Invoke-Tests.ps1
$LASTEXITCODE
```

Success means exit code is 0 .

If not 0:stop and escalate.

Step C - Run preflight readiness check

Where: the machine that will run verification/deploy.

```
.\Invoke-Preflight.ps1 -Processor <system>,
    -ApprovedCommitParam <approvedCommitParam>'
```



```
-TrustParam <trustParam> -  
-ManifestPath <manifestPath>
```

\$LASTEXITCODE

Success means exit code is 0 .

If not 0 :stop and fix access/paths before moving on.

Step D - Capture approved UAT baseline (approval required)

Where: UAT approval host.

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -  
-File .\Invoke-Verification.psl -  
-Mode Capture -  
-ReleaseRoot <releaseRoot> -  
-ManifestPath <manifestPath> -  
-TrustParam <trustParam> -  
-ArchiveRepo <repoRoot> -  
-ReleaseTag <releaseTag> -  
-Processor <system> -  
-CommitSha (git rev-parse HEAD) -  
-Environment uat -  
-Region <region> -  
-Json  
$LASTEXITCODE
```

Success means exit code is 0 .

If not 0 :stop and escalate.

Step E - Verify production files against approved baseline

Where: production target server.

```
powershell.exe -NoProfile -ExecutionPolicy Bypass'  
-File .\Invoke-Verification.psl -  
-Mode VerifyFiles'  
-ReleaseRoot <releaseRoot>'  
-ManifestPath <manifestPath>'  
-TrustParam <trustParam>'  
-Processor <system> -  
-Environment prod'  
-Region <region>'  
-Json
```

\$LASTEXITCODE

Success means:

- Exit code 0
- No MISSING , CHANGED , or EXTRA entries

If not 0 :stop and escalate with logs.

Step F - Verify config {if in scope}

Where: production target server.

```
.\Invoke-Verification.psl -Mode VerifyConfig -Processor <system>  
$LASTEXITCODE
```

Success means exit code 0 .

If not 0 :stop and escalate.

Step G - Run health check after deploy/verify

Where: production target server.

```

.\Invoke-HealthCheck.ps1 -Processor <system>'
  -ScheduledTasks <taskName>,
  -ProcessPathRoot <releaseRoot>'
  -FreshlogDir <logDir>
$LASTEXITCODE

```

Success means exit code 0 .

If exit code is 3: health failed - escalate immediately.

5) Optional deployment steps

Use these only during an approved change window.

1. Pre-deploy gate on staging/deploy machine:

```

.\Invoke-PreDeployGate.ps1 -StagedRoot <stagedRoot> -StagedCommit <sha>,
  -ApprovedCommitParam <approvedCommitParam>'
  -BaselineRepo <repoRoot> -ReleaseTag <releaseTag>'
  -TrustParam <trustParam> -Processor <system>
$LASTEXITCODE

```

2. Dry-rundeploy first on target server:

```

.\processors\Deploy-<system>.ps1 -StagedRoot <stagedRoot> -StagedCommit <sha>
$LASTEXITCODE

```

3. Real deploy on target server:

```

.\processors\Deploy-<system>.ps1 -StagedRoot <stagedRoot> -StagedCommit <sha>
$LASTEXITCODE

```

6) Exit code quick help

Exit Code	Plain meaning	What to do
0	Success	Record result and continue
1	Files/config do not match approved baseline	Stop and escalate
2	Trust/access/runtime issue (often \$SM/path/access)	Stop, validate inputs/access, escalate
3	Health check failed	Stop and escalate immediately
10	Invalid or unsafe usage/input	Correct command inputs and rerun

7) What to record for sign-off

For every step, record:

- Date/time
- Server name
- Script name
- Commit (`git rev-parse --short HEAD`)
- Exit code
- Log file path

Keep secrets out of notes and screenshots.

8) Escalation packet (send this when a step fails)

Include:

1. Script and command used
 2. Machine/server name
 3. Date/time
 4. Exit code
 5. Output lines showing failures (MISSING , CHANGED , EXTRA , or health errors)
 6. Log file path
-

9) Related guides

- Technical deep-dive runbook: [RUNBOOK.md](#)
- Tester-focused flow details: [SCRIPT-TESTING-GUIDE.md](#)
- Server/path reference: [SERVERS.md](#)
- Full behavior and trust model: [README.md](#)